Artificial Intelligence Technology and Application

Deep Learning Lab Guide

Student Version

HUAWEI

HUAWEI TECHNOLOGIES CO., LTD.

About This Document

Description

This guide introduces the following five exercises:

- Exercise 1: MindSpore basics, which describes the basic syntax and common modules of MindSpore.
- Exercise 2: Handwritten character recognition, in which the MindSpore framework is used to recognize handwritten characters.
- Exercise 3: MobileNetV2 image classification, which mainly introduces the classification of flower images using the lightweight network MobileNetV2.
- Exercise 4: ResNet-50 image classification exercise, which mainly introduces the classification of flower images using the ResNet-50 model.
- Exercise 5: TextCNN sentiment analysis, which mainly introduces sentiment analysis of statements using the TextCNN model.

Background Knowledge Required

This course is for Huawei's basic certification. To better understand this course, familiarize yourself with the following:

- Basic knowledge of Python, MindSpore concepts, and Python programming.

Contents

About This Document 1
1 MindSpore Basics 3
1.1 Introduction 3
1.1.1 About This Lab 3
1.1.2 Objectives 3
1.1.3 Lab Environment 3
1.2 Procedure 3
1.2.1 Introduction to Tensors 3
1.2.2 Loading a Dataset 8
1.2.3 Building the Network 11
1.2.4 Training and Validating a Model 14
1.2.5 Saving and Loading a Model 15
1.2.6 Automatic Differentiation 16
1.3 Question 18

1 MindSpore Basics

1.1 Introduction

1.1.1 About This Lab

This exercise introduces the tensor data structure of MindSpore. By performing a series of operations on tensors, you can understand the basic syntax of MindSpore.

1.1.2 Objectives

- Master the method of creating tensors.
- Master the attributes and methods of tensors.

1.1.3 Lab Environment

MindSpore 1.7 or later is recommended. The exercise can be performed on a PC or by logging in to HUAWEI CLOUD and purchasing the ModelArts service.

1.2 Procedure

1.2.1 Introduction to Tensors

Tensor is a basic data structure in MindSpore network computing. For details about data types in tensors, see the dtype description on the MindSpore official website.

Tensors of different dimensions represent different data. For example, a 0-dimensional tensor represents a scalar, a 1-dimensional tensor represents a vector, a 2-dimensional tensor represents a matrix, and a 3-dimensional tensor may represent the three channels of RGB images.

MindSpore tensors support different data types, including int8, int16, int32, int64, uint8, uint16, uint32, uint64, float16, float32, float64 and bool_, which correspond to the data types of NumPy.

In the computation process of MindSpore, the int data type in Python is converted into the defined int64 type, and the float data type is converted into the defined float32 type.

1.2.1.1 Creating a Tensor

During tensor construction, the tensor, float, int, Boolean, tuple, list, and NumPy.array types can be input. The tuple and list can store only data of the float, int, and Boolean types.

The data type can be specified during tensor initialization. However, if the data type is not specified, the initial values **int**, **float**, and **bool** generate 0-dimensional tensors with mindspore.int32, mindspore.float32 and mindspore.bool_ data types, respectively. The data types of the 1-dimensional tensors generated by the initial values **tuple** and **list** correspond to the data types of tensors stored in the tuple and list. If multiple types of data are contained, the MindSpore data type corresponding to the data type with the highest priority is selected (Boolean < int < float). If the initial value is **Tensor**, the data type is tensor. If the initial value is **NumPy.array**, the generated tensor data type corresponds to NumPy.array.

Step 1 Create a tensor using an array.

Code:

```
# Import MindSpore.
import mindspore
# The cell outputs multiple lines at the same time.
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"

import numpy as np
from mindspore import Tensor
from mindspore import dtype
# Use an array to create a tensor.
x = Tensor(np.array([[1, 2], [3, 4]]), dtype.int32)
x
```

Output:

```
Tensor(shape=[2, 2], dtype=Int32, value=
[[1, 2],
 [3, 4]])
```

Step 2 Create tensors using numbers.

Code:

```
# Use a number to create tensors.
y = Tensor(1.0, dtype.int32)
z = Tensor(2, dtype.int32)
y
z
```

Output:

```
Tensor(shape=[], dtype=Int32, value= 1)
Tensor(shape=[], dtype=Int32, value= 2)
```

Step 3 Create a tensor using Boolean.

Code:

```
# Use Boolean to create a tensor.
```

```
m = Tensor(True, dtype.bool_)
m
```

Output:

```
Tensor(shape=[], dtype=Bool, value= True)
```

Step 4 Create a tensor using a tuple.

Code:

```
# Use a tuple to create a tensor.
```

Output

```
Tensor(shape=[3], dtype=Int16, value= [1, 2, 3])
```

Step 5 Create a tensor using a list.

Code:

```
# Use a list to create a tensor.
```

Output:

```
Tensor(shape=[3], dtype=Float64, value= [4.00000000e+000, 5.00000000e+000, 6.00000000e+000]
```

Step 6 Inherit attributes of another tensor to form a new tensor.

Code:

```
from mindspore import ops
oneslike = ops.OnesLike()
x = Tensor(np.array([[0, 1], [2, 1]]).astype(np.int32))
output = oneslike(x)
output
```

Output:

```
Tensor(shape=[2, 2], dtype=Int32, value=
[[1, 1],
 [1, 1]])
```

Step 7 Output constant tensor value.

Code:

```
from mindspore.ops import operations as ops

shape = (2, 2)
ones = ops.Ones()
output = ones(shape,dtype.float32)
print(output)

zeros = ops.Zeros()
output = zeros(shape, dtype.float32)
print(output)
```

Output:

```
[[1. 1.]
 [1. 1.]]
[[0. 0.]
 [0. 0.]]
```

1.2.1.2 Tensor Attributes

Tensor attributes include shape and data type (dtype).

- Shape: a tuple
- Data type: a data type of MindSpore

Code:

```
x = Tensor(np.array([[1, 2], [3, 4]]), dtype.int32)

# Shape

# Data type

# Dimension

# Size
```

Output:

```
(2, 2)
mindspore.int32
2
4
```

1.2.1.3 Tensor Methods

asnumpy(): converts a tensor to an array of NumPy.

Code:

```
y = Tensor(np.array([[True, True], [False, False]]), dtype.bool_)

# Convert the tensor data type to NumPy.
```

```
```

Output:

```
Tensor(shape=[2, 2], dtype=Bool, value=
[[ True,  True],
 [False, False]])

array([[ True,  True],
       [False, False]])
```

1.2.1.4 Tensor Operations

There are many operations between tensors, including arithmetic, linear algebra, matrix processing (transposing, indexing, and slicing), and sampling. The following describes several operations. The usage of tensor computation is similar to that of NumPy.

Step 1 Perform indexing and slicing.

Code:

```
tensor = Tensor(np.array([[0, 1], [2, 3]]).astype(np.float32))
print("First row: {}".format(tensor[0]))
print("First column: {}".format(tensor[:, 0]))
print("Last column: {}".format(tensor[..., -1]))
```

Output:

```
First row: [0. 1.]
First column: [0. 2.]
Last column: [1. 3.]
```

Step 2 Concatenate tensors.

Code:

```
data1 = Tensor(np.array([[0, 1], [2, 3]]).astype(np.float32))
data2 = Tensor(np.array([[4, 5], [6, 7]]).astype(np.float32))
op = ops.Stack()
output = op([data1, data2])
print(output)
```

Output:

```
[[[0. 1.]
  [2. 3.]]

 [[4. 5.]
  [6. 7.]]]
```

Step 3 Convert to NumPy.

Code:

```
#
```

Output:

```
output: <class 'mindspore.common.tensor.Tensor'>
n_output: <class 'numpy.ndarray'>
```

1.2.2 Loading a Dataset

MindSpore.dataset provides APIs to load and process datasets such as MNIST, CIFAR-10, CIFAR-100, VOC, ImageNet, and CelebA.

Step 1 Load the MNIST dataset.

You are advised to download the MNIST dataset from https://certification-data.obs.cn-north-4.myhuaweicloud.com/CHS/HCIA-AI/V3.5/chapter4/MNIST.zip and save the training and test files to the MNIST folder.

Code:

```
import os
import mindspore.dataset as ds
import matplotlib.pyplot as plt

dataset_dir = "./MNIST/train"    # Path of the dataset
# Read three images from the MNIST dataset.
mnist_dataset = ds.MnistDataset(dataset_dir=dataset_dir, num_samples=3)
# View the images and set the image sizes.
plt.figure(figsize=(8,8))
i = 1

# Print three subgraphs.
```

Output:

Figure 1-1 MNIST dataset sample

Step 2 Customize a dataset.

For datasets that cannot be directly loaded by MindSpore, you can build a custom dataset class and use the GeneratorDataset API to customize data loading.

Code:

```
import numpy as np
np.random.seed(58)

class DatasetGenerator:
# When a dataset object is instantiated, the __init__ function is called. You can perform operations such as data initialization.
    def __init__(self):
        self.data = np.random.sample((5, 2))
        self.label = np.random.sample((5, 1))
# Define the __getitem__ function of the dataset class to support random access and obtain and
#return data in the dataset based on the specified index value.



# Define the __len__ function of the dataset class and return the number of samples in the dataset.



# After the dataset class is defined, the GeneratorDataset API can be used to load and access dataset



# Use the create_dict_iterator method to obtain data.
```

Output:

```
[0.36510558 0.45120592] [0.78888122]
[0.49606035 0.07562207] [0.38068183]
[0.57176158 0.28963401] [0.16271622]
[0.30880446 0.37487617] [0.54738768]
[0.81585667 0.96883469] [0.77994068]
```

Step 3 Perform data augmentation.

The dataset APIs provided by MindSpore support data processing methods such as shuffle and batch. You only need to call the corresponding function API to quickly process data.

In the following example, the datasets are shuffled, and two samples form a batch.

Code:

```
ds.config.set_seed(58)

# Shuffle the data sequence. buffer_size indicates the size of the shuffled buffer in the dataset.
```

```
# Divide the dataset into batches. batch_size indicates the number of data records contained in each
```

Output:

```
data: [[0.36510558 0.45120592]
  [0.57176158 0.28963401]]
label: [[0.78888122]
  [0.16271622]]
data: [[0.30880446 0.37487617]
  [0.49606035 0.07562207]]
label: [[0.54738768]
  [0.38068183]]
data: [[0.81585667 0.96883469]]
label: [[0.77994068]]
```

Code:

```
import matplotlib.pyplot as plt

from mindspore.dataset.vision import Inter
import mindspore.dataset.vision.c_transforms as c_vision

DATA_DIR = './MNIST/train'
# Obtain six samples.


# View the original image data.
```

Output:

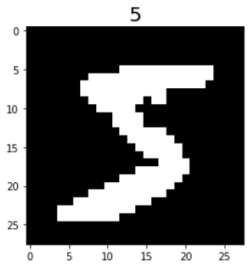

Figure 1-2 Data sample

Code:

```
resize_op = c_vision.Resize(size=(40,40), interpolation=Inter.LINEAR)
crop_op = c_vision.RandomCrop(28)
transforms_list = [resize_op, crop_op]
mnist_dataset = mnist_dataset.map(operations=transforms_list, input_columns=["image"])
mnist_dataset = mnist_dataset.create_dict_iterator()
data = next(mnist_dataset)
plt.imshow(data['image'].asnumpy().squeeze(), cmap=plt.cm.gray)
plt.title(data['label'].asnumpy(), fontsize=20)
plt.show()
```

Output:

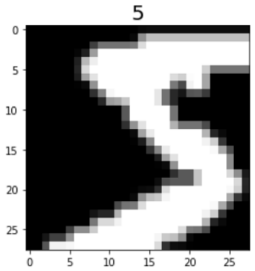

Figure 1-3 Effect after data argumentation

1.2.3 Building the Network

MindSpore encapsulates APIs for building network layers in the nn module. Different types of neural network layers are built by calling these APIs.

Step 1 Build a fully-connected layer.

Fully-connected layer: mindspore.nn.Dense

- **in_channels**: input channel
- **out_channels**: output channel
- **weight_init**: weight initialization. Default value: **'normal'**.

Code:

```
import mindspore as ms
import mindspore.nn as nn
from mindspore import Tensor
import numpy as np

# Construct the input tensor.
input_a = Tensor(np.array([[1, 1, 1], [2, 2, 2]]), ms.float32)
print(input_a)
# Construct a fully-connected network. Set both in_channels and out_channels to 3.
net = nn.Dense(in_channels=3, out_channels=3, weight_init=1)
output = net(input_a)
print(output)
```

Output:

```
[[1. 1. 1.]
```

```
  [2. 2. 2.]]
[[3. 3. 3.]
  [6. 6. 6.]]
```

Step 2 Build a convolutional layer.

Code:

```
conv2d = nn.Conv2d(1, 6, 5, has_bias=False, weight_init='normal', pad_mode='valid')
input_x = Tensor(np.ones([1, 1, 32, 32]), ms.float32)

print(conv2d(input_x).shape)
```

Output:

```
(1, 6, 28, 28)
```

Step 3 Build a ReLU layer.

Code:

```
relu = nn.ReLU()
input_x = Tensor(np.array([-1, 2, -3, 2, -1]), ms.float16)
output = relu(input_x)

print(output)
```

Output:

```
[0. 2. 0. 2. 0.]
```

Step 4 Build a pooling layer.

Code:

```
max_pool2d = nn.MaxPool2d(kernel_size=2, stride=2)
input_x = Tensor(np.ones([1, 6, 28, 28]), ms.float32)

print(max_pool2d(input_x).shape)
```

Output:

```
(1, 6, 14, 14)
```

Step 5 Build a Flatten layer.

Code:

```
flatten = nn.Flatten()
input_x = Tensor(np.ones([1, 16, 5, 5]), ms.float32)
output = flatten(input_x)

print(output.shape)
```

Output:

```
(1, 400)
```

Step 6 Define a model class and view parameters.

The Cell class of MindSpore is the base class for building all networks and the basic unit of a network. When a neural network is required, you need to inherit the Cell class and overwrite the __init__ and construct methods.

Code:

```
class LeNet5(nn.Cell):
    """
    Lenet network structure
    """

# Instantiate the model and use the parameters_and_names method to view the model parameters.
```

Output:

```
('conv1.weight', Parameter (name=conv1.weight, shape=(6, 1, 5, 5), dtype=Float32,
requires_grad=True))
('conv2.weight', Parameter (name=conv2.weight, shape=(16, 6, 5, 5), dtype=Float32,
requires_grad=True))
('fc1.weight', Parameter (name=fc1.weight, shape=(120, 400), dtype=Float32, requires_grad=True))
('fc1.bias', Parameter (name=fc1.bias, shape=(120,), dtype=Float32, requires_grad=True))
('fc2.weight', Parameter (name=fc2.weight, shape=(84, 120), dtype=Float32, requires_grad=True))
('fc2.bias', Parameter (name=fc2.bias, shape=(84,), dtype=Float32, requires_grad=True))
('fc3.weight', Parameter (name=fc3.weight, shape=(10, 84), dtype=Float32, requires_grad=True))
('fc3.bias', Parameter (name=fc3.bias, shape=(10,), dtype=Float32, requires_grad=True))
```

1.2.4 Training and Validating a Model

Step 1 Use loss functions.

A loss function is used to validate the difference between the predicted and actual values of a model. Here, the absolute error loss function L1Loss is used. mindspore.nn.loss also provides many other loss functions, such as SoftmaxCrossEntropyWithLogits, MSELoss, and SmoothL1Loss.

The output value and target value are provided to compute the loss value. The method is as follows:

Code:

```
import numpy as np
import mindspore.nn as nn
from mindspore import Tensor
import mindspore.dataset as ds
import mindspore as ms
loss = nn.L1Loss()
output_data = Tensor(np.array([[1, 2, 3], [2, 3, 4]]).astype(np.float32))
target_data = Tensor(np.array([[0, 2, 5], [3, 1, 1]]).astype(np.float32))
print(loss(output_data, target_data))
```

Output:

```
1.5
```

Step 2 Use an optimizer.

Common deep learning optimization algorithms include SGD, Adam, Ftrl, lazyadam, Momentum, RMSprop, Lars, Proximal_ada_grad, and lamb.

Momentum optimizer: mindspore.nn.Momentum

Code:

```
#
```

Step 3 Build a model.

mindspore.Model(network, loss_fn, optimizer, metrics)

Code:

```
from mindspore import Model

# Define a neural network.
net = LeNet5()
# Define the loss function.
loss = nn.SoftmaxCrossEntropyWithLogits(sparse=True, reduction='mean')
```

```
# Define the optimizer.
optim = nn.Momentum(params=net.trainable_params(), learning_rate=0.1, momentum=0.9)
# Build a model.
model = Model(network = net, loss_fn=loss, optimizer=optim, metrics={'accuracy'})
```

Step 4 Train the model.

Code:

```
import mindspore.dataset.transforms.c_transforms as C
import mindspore.dataset.vision.c_transforms as CV
from mindspore.train.callback import  LossMonitor

DATA_DIR = './MNIST/train'
mnist_dataset = ds.MnistDataset(DATA_DIR)

resize_op = CV.Resize((28,28))
rescale_op = CV.Rescale(1/255,0)
hwc2chw_op = CV.HWC2CHW()

mnist_dataset  = mnist_dataset .map(input_columns="image", operations=[rescale_op,resize_op,
hwc2chw_op])
mnist_dataset  = mnist_dataset .map(input_columns="label", operations=C.TypeCast(ms.int32))
mnist_dataset = mnist_dataset.batch(32)
loss_cb = LossMonitor(per_print_times=1000)
# dataset is an input parameter, which indicates the training set, and epoch indicates the number of
training epochs of the training set.
model.train(epoch=1, train_dataset=mnist_dataset,callbacks=[loss_cb])
```

Step 5 Validate the model.

Code:

```
# Test set
DATA_DIR = './forward_mnist/MNIST/test'
dataset = ds.MnistDataset(DATA_DIR)

resize_op = CV.Resize((28,28))
rescale_op = CV.Rescale(1/255,0)
hwc2chw_op = CV.HWC2CHW()

dataset  = dataset .map(input_columns="image", operations=[rescale_op,resize_op, hwc2chw_op])
dataset  = dataset .map(input_columns="label", operations=C.TypeCast(ms.int32))
dataset = dataset.batch(32)
model.eval(valid_dataset=dataset)
```

1.2.5 Saving and Loading a Model

After the preceding network model is trained, the model can be saved in two forms:

1. One is to simply save the network model before or after training. The advantage is that the API is easy to use, but only the network model status when the command is executed is retained.

Code:

```
import mindspore as ms

# net indicates a defined network model, which is used before or after training.
ms.save_checkpoint(net, "./MyNet.ckpt") # net indicates the training network, and ./MyNet.ckpt
indicates the path for saving the network model.
```

Output:

```
epoch: 1 step: 1000, loss is 2.283555746078491
```

2. The other one is to save the interface during network model training. MindSpore automatically saves the number of epochs and number of steps set during training. That is, the intermediate weight parameters generated during the model training process are also saved to facilitate network fine-tuning and stop training.

Code:

```
from mindspore.train.callback import ModelCheckpoint, CheckpointConfig

# Set the value of epoch_num.
epoch_num = 5

# Set model saving parameters.
config_ck = CheckpointConfig(save_checkpoint_steps=1875, keep_checkpoint_max=10)

# Use model saving parameters.
ckpoint = ModelCheckpoint(prefix="lenet", directory="./lenet", config=config_ck)
model.train(epoch_num, mnist_dataset, callbacks=[ckpoint])
```

1.2.6 Automatic Differentiation

Backward propagation is the commonly used algorithm for training neural networks. In this algorithm, parameters (model weights) are adjusted based on a gradient of a loss function for a given parameter.

The first-order derivative method of MindSpore is **mindspore.ops.GradOperation (get_all=False, get_by_list=False, sens_param=False)**. When **get_all** is set to **False**, the first input derivative is computed. When **get_all** is set to **True**, all input derivatives are computed. When **get_by_list** is set to **False**, weight derivatives are not computed. When **get_by_list** is set to **True**, weight derivatives are computed. **sens_param** scales the output value of the network to change the final gradient.

The following uses the MatMul operator derivative for in-depth analysis.

Step 1 Compute the first-order derivative of the input.

To compute the input derivative, you need to define a network requiring a derivative. The following uses a network **f(x,y)=z*x*y** formed by the MatMul operator as an example.

Code:

```
import numpy as np
import mindspore.nn as nn
import mindspore.ops as ops
from mindspore import Tensor
```

```
from mindspore import ParameterTuple, Parameter
from mindspore import dtype as mstype
```

Output:

```
[[4.5099998 2.7         3.6000001]
 [4.5099998 2.7         3.6000001]]
```

Step 2 Compute the first-order derivative of the weight.

To compute weight derivatives, you need to set **get_by_list** in **ops.GradOperation** to **True**. If computation of certain weight derivatives is not required, set **requires_grad** to **False** when you definite the network.

Code:

```
#
```

Output:

```
(Tensor(shape=[1], dtype=Float32, value= [ 2.15359993e+01]),)
```

1.3 Question

When the following code is used to create two tensors, t1 and t2, can t1 be created properly? Check whether the two tensors have the same outputs. If not, what is the difference?